

BIO310 – Lab 6

Ekin Renas Katirci/ID:31302

Task 1: de Bruijn Graphs

1. de Bruijn Graph from 3-mer Composition

Given the 3-mer composition:

ABO, ALL, BAB, BAL, BOO, BOO, LLO, LOO, NBA, NBA, ONB, ONB, OON, OON,
OON

In a de Bruijn graph, each k -mer becomes a directed edge. The source node is the first $(k-1)$ characters and the target node is the last $(k-1)$ characters. For $k = 3$, the edge table is:

3-mer	Source (prefix)	Target (suffix)
ABO	AB	BO
ALL	AL	LL
BAB	BA	AB
BAL	BA	AL
BOO ×2	BO	OO
LLO	LL	LO
LOO	LO	OO
NBA ×2	NB	BA
ONB ×2	ON	NB
OON ×3	OO	ON

The resulting de Bruijn graph is shown in Figure 1.

1(a). Finding All Possible Sequences (Eulerian Paths)

To find the original sequences, we need to find all Eulerian paths in the graph. First, we compute in-degree and out-degree for each node:

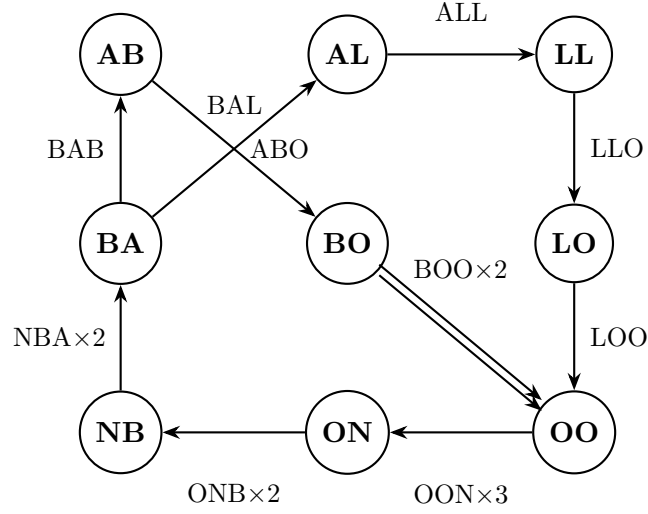


Figure 1: de Bruijn graph constructed from the given 3-mer composition (also the $k = 3$ graph for both sequences). Edge multiplicities are shown where applicable.

Node	In-degree	Out-degree	Difference
AB	1	1	0
AL	1	1	0
LL	1	1	0
BA	2	2	0
BO	1	2	+1
LO	1	1	0
NB	2	2	0
ON	3	2	-1
OO	3	3	0

Since BO has out-degree $>$ in-degree by 1, it is the **start node**. Since ON has in-degree $>$ out-degree by 1, it is the **end node**. Two distinct Eulerian paths exist, differing at the branching point at node BA:

Path 1:

BO $\rightarrow$ OO $\rightarrow$ ON $\rightarrow$ NB $\rightarrow$ BA $\rightarrow$ AB $\rightarrow$ BO $\rightarrow$ OO $\rightarrow$ ON $\rightarrow$ NB $\rightarrow$ BA $\rightarrow$ AL $\rightarrow$ LL $\rightarrow$ LO $\rightarrow$ OO $\rightarrow$ ON

Sequence: BOONBABOONBALLOON

Path 2:

BO $\rightarrow$ OO $\rightarrow$ ON $\rightarrow$ NB $\rightarrow$ BA $\rightarrow$ AL $\rightarrow$ LL $\rightarrow$ LO $\rightarrow$ OO $\rightarrow$ ON $\rightarrow$ NB $\rightarrow$ BA $\rightarrow$ AB $\rightarrow$ BO $\rightarrow$ OO $\rightarrow$ ON

Sequence: BOONBALLOONBABOON

2(a). de Bruijn Graphs for Each Sequence with $k = 3$

We extract the 3-mer composition from each sequence by sliding a window of size 3:

Sequence 1 – BOONBABOONBALLOON:

BOO, OON, ONB, NBA, BAB, ABO, BOO, OON, ONB, NBA, BAL, ALL, LLO, LOO, OON

Sequence 2 – BOONBALLOONBABOON:

BOO, OON, ONB, NBA, BAL, ALL, LLO, LOO, OON, ONB, NBA, BAB, ABO, BOO, OON

Sorting and counting the 3-mers from both sequences:

3-mer	Count in Seq. 1	Count in Seq. 2
ABO	1	1
ALL	1	1
BAB	1	1
BAL	1	1
BOO	2	2
LLO	1	1
LOO	1	1
NBA	2	2
ONB	2	2
OON	3	3

Both sequences produce **exactly the same** 3-mer multiset, and therefore generate **identical** de Bruijn graphs (the same graph shown in Figure 1). Consequently, the two sequences **cannot be distinguished** at $k = 3$.

2(b). de Bruijn Graphs with $k = 5$

For $k = 5$, each 5-mer becomes an edge with a 4-character source and a 4-character target node.

Sequence 1 – BOONBABOONBALLOON (13 edges):

BOONB, OONBA, ONBAB, NBABO, BABOO, ABOON, BOONB, OONBA, ONBAL, NBALL, BALLO, ALLOO, LLOON

Sequence 2 – BOONBALLOONBABOON (13 edges):

BOONB, OONBA, ONBAL, NBALL, BALLO, ALLOO, LLOON, LOONB, OONBA, ONBAB, NBABO, BABOO, ABOON

Comparing the 5-mer multisets:

5-mer	Count in Seq. 1	Count in Seq. 2
BOONB	2	1
OONBA	2	2
ONBAB	1	1
NBABO	1	1
BABOO	1	1
ABOON	1	1
ONBAL	1	1
NBALL	1	1
BALLO	1	1
ALLOO	1	1
LLOON	1	1
LOONB	0	1

The critical difference: Sequence 1 has the edge $\text{BOON} \rightarrow \text{OONB}$ appearing **twice** ($\text{BOONB} \times 2$), whereas Sequence 2 has it **once** and instead includes an additional edge $\text{LOON} \rightarrow \text{OONB}$ (from the 5-mer LOONB). The full $k = 5$ graphs are shown in Figures 2 and 3.

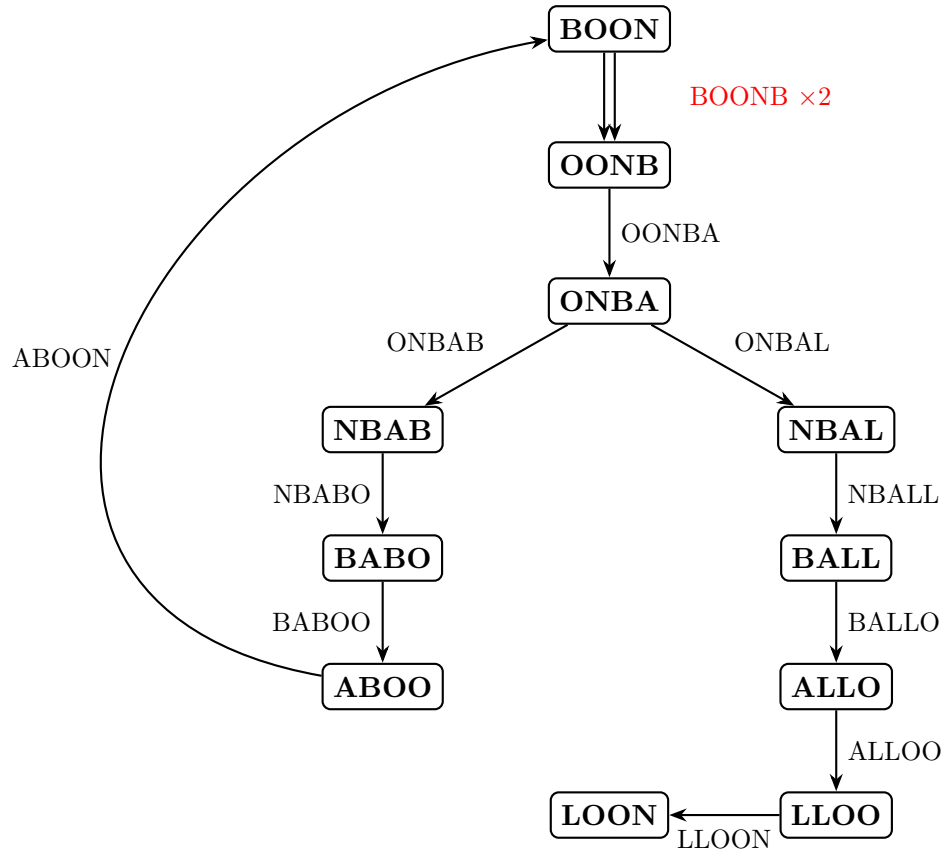


Figure 2: $k = 5$ de Bruijn graph for Sequence 1 (BOONBABOONBALLOON). The edge $\text{BOON} \rightarrow \text{OONB}$ appears **twice** (double arrow). LOON has no outgoing edge to OONB .

Since the graphs are different, each graph has a **unique** Eulerian path. Therefore, the two sequences **can be distinguished** at $k = 5$.

However, increasing k too much can also be problematic in real genomic data, as it requires longer reads and may fragment the graph if coverage is insufficient.

Task 2: Genome Assembly of *Veysonella kayaomans*

Implementation

Three functions were implemented to complete the genome assembly pipeline:

1. `rc(seq)`: Returns the reverse complement of a DNA string by reversing the sequence and replacing each base with its complement ($A \leftrightarrow T$, $C \leftrightarrow G$).
2. `getkmers(seq, k)`: Extracts all k -mers from a sequence by sliding a window of size k across the string, returning a list of substrings.
3. `DeBruijn(kmers)`: Constructs a de Bruijn graph from a list of k -mers. Each k -mer is split into a prefix (first $k-1$ characters) and suffix (last $k-1$ characters). The prefix becomes the source node and the suffix becomes the target node, stored as an adjacency list dictionary.

Results

The `genomeAssembly` function iterates from the largest possible k downward until it finds a value that produces a valid Eulerian circuit in the de Bruijn graph. The assembled genome has the following properties:

Parameter	Value
Assembled genome length	2000 bp
k -mer size used	$k = 37$
Number of input reads	2001 (including reverse complements)
Read length	50 bp

Colab Output

The output of the `genomeAssembly` function is a tuple (`genome`, `k`, `len(genome)`):

```
('TGCAAACCGCACAGAATAGAATGGTAGACCGTGTCGGATTGTACACTACATA...  
...[1900+ bases omitted for brevity]...  
...CAACTGGCAGCTCCGGTATCACGAAGGGCTGCGG', 37, 2000)
```

The algorithm successfully reconstructed the circular genome of *Veysonella kayaomans* by finding an Eulerian circuit in the de Bruijn graph constructed from 37-mers derived from both the original reads and their reverse complements. The assembled genome is **2000 bp** long and was assembled using $k = 37$.